



BIRZEIT UNIVERSITY

Department of Electrical and Computer Engineering

Summer Semester, 2025-2026

Second Assignment, Due: September 24, 2026

Arabic Sentiment Analysis

1. Description

This project aims to conduct a comprehensive study of sentiment analysis techniques for Arabic social media data by investigating different text representation methods and classification models. The primary focus is to analyze how various feature representations, including traditional and embedding-based approaches, influence sentiment classification performance, and to compare the effectiveness of non-sequential and sequential deep learning architectures.

Students will explore multiple word representation techniques such as TF-IDF, Word2Vec, FastText, and transformer-based embeddings, and evaluate their impact when combined with different machine learning and deep learning classifiers. The project also emphasizes systematic hyperparameter optimization and rigorous experimental evaluation to identify the most effective combinations of representations and models for Arabic sentiment analysis.

In addition, the project asks students to go beyond a single best-model search: to examine how their preprocessing choices, data augmentation, and class-imbalance handling strategies affect model performance, and to justify their design decisions with evidence from controlled experiments.

2. Objectives

The primary objectives of this project are:

1. To study and compare different text representation techniques for Arabic sentiment analysis, including:
 - TF-IDF representations
 - Word2Vec embeddings
 - FastText embeddings
 - Contextual embeddings derived from transformer-based language models (optional)
2. To investigate the effectiveness of different classification paradigms by comparing:
 - Traditional machine learning classifiers such as Naïve Bayes and Random Forests.
 - Non-sequential deep learning models such as Convolutional Neural Networks (CNNs).
 - Sequential deep learning models such as Recurrent Neural Networks (RNNs) and Long Short-Term Memory Networks (LSTMs).
3. To perform systematic hyperparameter optimization for all models and analyze the effect of model parameters on classification performance.
4. To evaluate and compare all models using standard sentiment analysis metrics including Accuracy, Precision, Recall, and F1-Score.
5. To identify the most effective combinations of text representation methods and classification models for Arabic sentiment classification and provide insights into their strengths and limitations.

3. Dataset Overview

The project will utilize the ArSarcasm-v2 dataset, a collection of 15,548 Arabic tweets (12,548 for training and 3,000 for testing) originally released for the WANLP 2021 shared task on sarcasm and sentiment detection. Each tweet is

annotated with a sentiment label (Positive, Negative, or Neutral); this project's classification task is built on the sentiment label.

In addition to sentiment, each tweet also carries a sarcasm label (True/False) and a dialect label (MSA, Egyptian, Gulf, Levantine, or Maghrebi). These auxiliary annotations are not required for the core sentiment classification task, but they are the basis for the class-imbalance handling, preprocessing ablation, and data augmentation tasks described below, and may optionally be used for further error analysis (e.g., examining whether sarcastic tweets or specific dialects are harder to classify correctly).

The dataset presents several challenges that must be carefully addressed during model development and evaluation, including:

- Class imbalance in the sentiment labels, and more pronounced imbalance in the sarcasm label (roughly 1 sarcastic tweet for every 5 non-sarcastic ones) and the dialect label (MSA dominates, while Maghrebi tweets are very rare).
- Linguistic noise such as spelling variations, informal expressions, emojis, and dialect-specific vocabulary.
- A correlation between sarcasm and negative sentiment, which can make sarcastic tweets a useful lens for error analysis.

4. Tasks and Methodology

1. Data Preprocessing

Perform appropriate Arabic text preprocessing, which may include:

- Text cleaning and normalization.
- Removal of URLs, HTML tags, punctuation, and unnecessary symbols.
- Arabic character normalization.
- Stop-word removal.
- Stemming or lemmatization.
- Handling emojis, hashtags, and negation expressions.

Students should justify the preprocessing pipeline based on the selected representation methods and experimental objectives.

2. Preprocessing Ablation Study

Conduct an ablation study to quantify the individual contribution of each preprocessing step to classification performance. Using a fixed representation-model combination, students should systematically enable/disable one preprocessing step at a time (e.g., with vs. without stop-word removal, with vs. without stemming/lemmatization, with vs. without emoji/hashtag handling) and measure the resulting change in performance.

The ablation study should include:

- A clearly defined baseline pipeline (full preprocessing) and a set of ablated variants (one step removed at a time).
- A results table reporting the performance delta (e.g., change in F1-Score) introduced by each step.
- A discussion of which preprocessing steps matter most for this dataset, and why.

3. Data Augmentation

Given the class imbalance present in the dataset, apply at least one Arabic-appropriate data augmentation technique to enrich the minority sentiment classes, such as:

- Back-translation (e.g., Arabic $\rightarrow$ English $\rightarrow$ Arabic).
- Synonym replacement using an Arabic lexical resource (e.g., Arabic WordNet).

- Contextual augmentation using a pretrained Arabic language model (e.g., masked-word prediction with AraBERT).
- Easy Data Augmentation (EDA)-style operations (random insertion, deletion, or swap) adapted for Arabic text.

Students should compare model performance with and without the augmented data, and report the effect on the minority class(es) specifically, not only on overall accuracy.

4. Feature Representation and Embedding Analysis

Implement and compare multiple text representation approaches, including:

- TF-IDF
- Word2Vec
- FastText
- Transformer-based embeddings (optional)

Analyze the advantages and limitations of each representation method and their impact on downstream classification performance.

5. Model Development and Training

Train and evaluate the following categories of classifiers:

A. Traditional Machine Learning Models

- Naïve Bayes
- Random Forests

B. Non-Sequential Deep Learning Models

- Convolutional Neural Networks (CNNs)

C. Sequential Deep Learning Models

- Recurrent Neural Networks (RNNs)
- Long Short-Term Memory Networks (LSTMs)

6. Dataset Splitting and Hyperparameter Optimization

Split the dataset into:

- 60% Training
- 20% Validation
- 20% Testing

Perform systematic hyperparameter tuning for each model.

7. Class-Imbalance Handling

Apply and compare at least one technique for handling class imbalance, such as:

- Class weighting / cost-sensitive learning.
- Oversampling of minority classes (e.g., SMOTE variants adapted for text) or undersampling of majority classes.
- Focal loss or other imbalance-aware loss functions for the deep learning models.

Report the effect of the chosen technique on minority-class performance (e.g., per-class Precision/Recall/F1) relative to the unweighted baseline, in addition to overall metrics.

8. Comparative Evaluation and Analysis

Evaluate all models using:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

Conduct a comparative analysis that investigates:

- The impact of different text representation methods.
- The performance differences between traditional machine learning and deep learning approaches.
- The performance differences between non-sequential models (CNN) and sequential models (RNN, LSTM).
- The influence of hyperparameter tuning on model performance.
- The findings of the preprocessing ablation study.
- The effect of data augmentation on minority-class performance.
- The effect of the class-imbalance handling technique on minority-class performance.
- The strengths and limitations of each representation-model combination.

5. Tools

Python is recommended as the primary programming language for this project. Students may use any suitable libraries and frameworks required to implement and evaluate their models. Commonly used libraries include:

- **Pandas** for data manipulation and analysis.
- **NumPy** for numerical computations.
- **NLTK, Camel Tools**, or similar libraries for Arabic text preprocessing.
- **Scikit-learn** for traditional machine learning models, feature extraction, and evaluation.
- **Gensim** for training and utilizing Word2Vec and FastText embeddings.
- **TensorFlow/Keras** or **PyTorch** for implementing deep learning models such as CNNs, RNNs, and LSTMs.
- **Matplotlib and Seaborn** for visualization and experimental analysis.

Students may use additional libraries when appropriate, provided they clearly document their usage.

6. Submission

Students are required to submit the following components:

1. Report (PDF Format)

The report should provide a comprehensive description of the entire project and must include the following:

- A clear problem formulation and project overview.
- A description of the dataset and any challenges associated with Arabic sentiment analysis.
- A detailed explanation of the preprocessing pipeline and the rationale behind the selected preprocessing techniques.
- The results and discussion of the preprocessing ablation study.
- A description of the data augmentation technique(s) applied and their impact on minority-class performance.
- A detailed description of each text representation method investigated, including TF-IDF, Word2Vec, FastText, and any additional embedding methods used.

- A description of all classification models implemented.
- A detailed explanation of the architecture and design choices for each deep learning model.
- A description of the hyperparameter optimization process.
- A description of the class-imbalance handling technique(s) applied and their effect on minority-class performance.
- Experimental results for all representation-model combinations.
- Performance evaluation using appropriate tables, graphs, and visualizations.
- A comparative analysis discussing all points listed in Section 4.8.
- A discussion of challenges encountered during the project and the strategies used to address them.

2. Source Code

- Submit all source code electronically. No hard copies are required.
- The code must be well-structured, modular, documented, and executable.
- The submission should include all scripts, notebooks, configuration files, and instructions necessary to reproduce the reported results.
- The code should clearly separate preprocessing, feature representation, data augmentation, model training, hyperparameter tuning, class-imbalance handling, and evaluation components.

Good Luck